The Journal of Open Source Software

DamageScanner: A Python package for natural hazard risk assessments

Elco E. Koks [1¶], **Hans de Moel** [1], **and Jens de Bruijn** [1]

1 Water & Climate Risk, Institute for Environmental Studies, Vrije Universiteit Amsterdam, The Netherlands ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Rachel Wegener

Reviewers:

- @abpoll
- @SouravDSGit

Submitted: 15 June 2026
Published: unpublished

Summary

The severity and frequency of natural hazards is increasing globally (IPCC, 2023). A good understanding of the risk to buildings and assets from hazards such as extreme precipitation, record-breaking temperatures, and severe landslides is essential to develop evidence-based risk-reducing measures. To do so, we require easy-to-use and hazard-agnostic tools that allow us to compute the natural hazard risk for any place in the world. Examples are a local rainfall event, or a large-scale earthquake event. While the magnitude and intensity of such events differ greatly, the computational workflow to estimate those impacts does not. As such, the `DamageScanner` has been developed as a simple and ready-to-use framework to allow exposure, damage and risk assessments either through user-defined input, or simply using public data as a starting point.

Statement of need

The `DamageScanner` is a Python package to perform exposure, damage and risk assessments for natural hazards and climate extremes. It focusses on making the calculations easy and computationally efficient. The exposure data can be both raster- and vector-based. The `DamageScanner` is designed to be small, supporting both direct use and integration in larger frameworks or models, supporting researchers and hazard risk modellers. It is also easily used by students in (introductory) courses on geospatial analysis and natural hazard risk modelling. It has already been used in a number of scientific publications (Koks et al., 2019; Koks & Haer, 2020), European projects (European Commission, 2023) and integrated into other python packages and models (e.g., `GEB` Bruijn et al. (2023)).

State of the field

Several open-source tools exist for natural hazard risk assessments, of which the `climada` framework (Aznar-Siguan & Bresch, 2019; Bresch & Aznar-Siguan, 2021) is one of the most well-known open-source tools. The `climada` framework is a widely-used Python ecosystem to perform large-scale risk assessment for various hazards. It allows for a multitude of use cases and contains a wide range of submodules to tailor specific needs within the natural hazard risk domain (Mühlhofer et al., 2023; Riedel et al., 2024). All of which is maintained by dedicated software engineers. While the `climada` framework is commendable and widely used, the large ecosystem and the many dependencies within the several submodules also make it less flexible for integration within other use cases.

As such, the `DamageScanner` has been continuously developed independently, for several reasons. Firstly, the `DamageScanner` has been developed organically to tailor specific use cases within several projects. For example, the original `DamageScanner` was built to perform raster-based

damage assessments, mostly using land use information, in combination with flood hazard maps (Klijn & Buren (2007); De Moel et al. (2012)). The current generation of the DamageScanner has, for example, been further developed to be integrated into several infrastructure and the built environment risk and resilience modelling pipelines, such as (Koks et al., 2019; Van Oosterhout et al., 2023; Ye et al., 2024).

Secondly, with the rise of publicly-available object-based data, in particular due to OpenStreetMap (OSM), a need arose to efficiently perform large-scale object-based damage calculations (Koks et al., 2019; Van Ginkel et al., 2021). These object-based damage and risk assessment are, however, much more computationally intensive due to the complexity of exact intersections of vector and raster data. The DamageScanner fills a gap here by providing an open-access approach to easily perform such risk assessment on top of existing object information (see the Software Design section for an explanation of this workflow).

Thirdly, geospatial information of natural hazards and climate extremes are stored in a range of different formats. Ranging from GeoTiffs and netCDFs, to vector-based representations of earthquake intensities and flood depths. To model the impacts across those hazards within the same modelling framework, a tool was needed that can readily read and use different data storage formats.

Software design

The DamageScanner is implemented in Python with a core design philosophy focused on simplicity. In its most simple form (as described in Figure 1), the user only needs to define four key inputs: (i) vector-based objects (e.g., buildings) or a raster-based land use map, (ii) a raster-based hazard-intensity map (e.g., a flood depth or wind speed), (iii) the maximum damages for each of the objects or land uses, and (iv) a set of vulnerability or fragility curves that describe the relation between the hazard intensity metric (e.g., the flood depth or the wind speed) and the potential impact to the specific objects or land uses in the applied assessment.

The Journal of Open Source Software

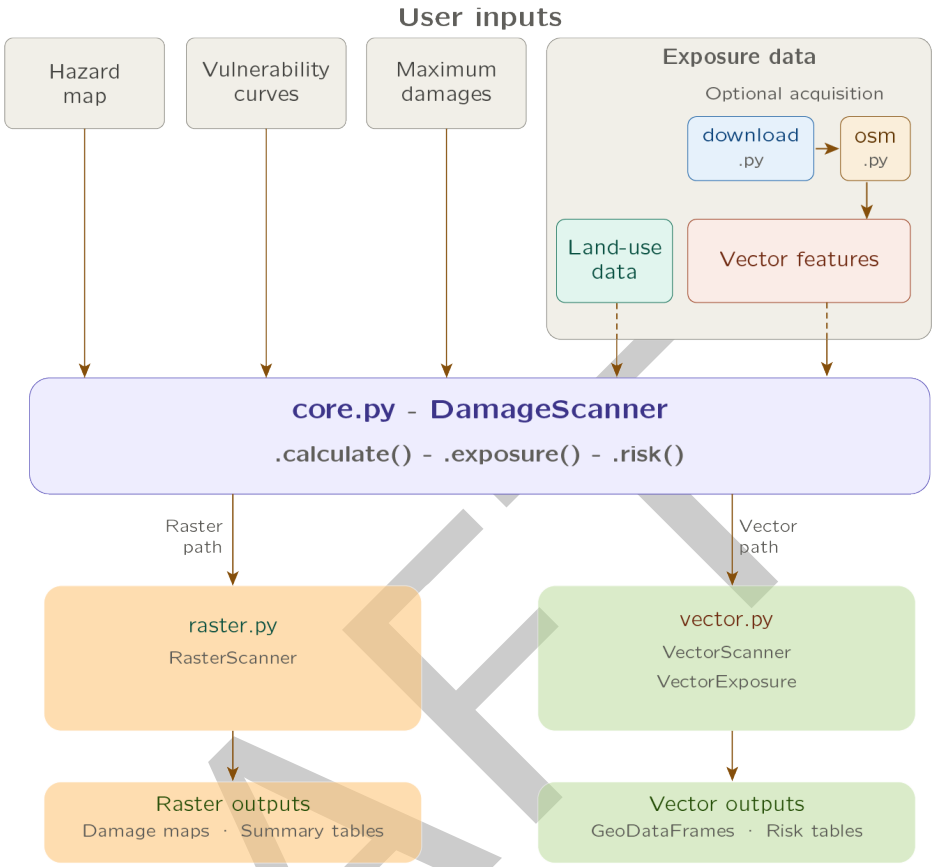

Figure 1: Software architecture of the DamageScanner package.

The `DamageScanner` can directly read most hazard maps using a provided file path (i.e., any format read by rasterio or xarray like GeoTiff or NetCDF), or they can be pre-loaded through the `xarray` package. Some examples from the public domain include JRC global flood maps (Baugh et al., 2024) and USGS ShakeMap. It is recommended to clip the hazard data to the area of interest before running the `DamageScanner`.

For the vulnerability curves, the `DamageScanner` can read all table-like data that can be read by pandas, such as csv- and Excel-files. Essential is the structure of the data. The first column should always represent the hazard intensity that corresponds with the hazard information (e.g., centimeters of inundation, maximum gust speeds in m/s). The following columns should then provide one or multiple vulnerability curves for each object/element considered in the analysis (e.g., different road, building or land-use types). Most vulnerability information is structured in such a manner (Nirandjan et al., 2024).

For the maximum damages, a file structure should be provided which links each object/element within the analysis to its maximum reconstruction/replacement value. The recommended structure is a left column with each object/element, and a right column with this maximum damage value.

The final piece of input information is the exposure data, either as raster or vector. Depending on the data type, the `DamageScanner` will perform a raster or a vector-based damage assessment. Given its wide-use in OSM-based damage and risk assessments, the `DamageScanner` allows to directly read OSM data through the `download.py` and `osm.py` scripts. Vector data can contain (Multi)Polygon, (Multi)LineString or Point geometries. It is important to ensure that the maximum damage values reflect these geometry types. More specifically, objects stored as Polygons will require maximum damage values in m2, objects stored as LineStrings will require

maximum damages values in meters, whereas objects stored as Points will require maximum damage values for the entire object.

Finally, if multiple return periods are available within the hazard data, one can estimate the potential natural hazard risk, which is computed as the area under the probability-exceedence loss curve through a trapezoid approach. The code-snippet below provides an example of the direct use of the `DamageScanner`.

```python
from damagescanner.core import DamageScanner

# Paths to input files
hazard = "path/to/hazard_data.tif"
feature_data = "path/to/exposure_data.shp"
curves = "path/to/vulnerability_curves.csv"
maxdam = "path/to/maxdam.csv"

# Initialize
scanner = DamageScanner(hazard, feature_data, curves, maxdam)

# Damage Assessment for single hazard
damage_results = scanner.calculate()

# Calculate risk using multiple hazard maps
hazard_dict = {
    10: "hazard_10.tif",
    50: "hazard_50.tif",
    100: "hazard_100.tif"
}

risk_results = scanner.risk(hazard_dict)

# Perform exposure-only calculation
# Curves and maxdam must be empty DataFrames for exposure-only
scanner = DamageScanner(hazard, feature_data, pd.DataFrame(), pd.DataFrame())
exposed = scanner.exposure()
```

When running the `DamageScanner`, it is important to be aware of several potential issues. The first, and most common issue when running the `DamageScanner` is consistency in the Coordinate Reference System (CRS). To reduce the risk of a potentially unsuccessful completion, it is recommended to ensure that both hazard and exposure information is stored in the same CRS. The second common issue relates to multiple geometries of the same object type. For example, when using OSM data, school facilities can be stored as both Points or Polygons. However, the maximum damages for a specific asset type can only refer to one geometry type (i.e., either damages per square meter, or per entire asset). As such, one either needs to convert all assets of the same object type into the same geometry type, or one should make unique object types for the different geometry types (and match maximum damages accordingly).

Research impact statement

The `DamageScanner` has demonstrated significant research impact and grown both its user base and contributor community since its initial release in 2019. It has been the basis of several academic papers (e.g., Koks et al. (2019); Van Oosterhout et al. (2023); Ye et al. (2024)) and European and international research projects (e.g., Horizon Europe MIRACA and Ticca4Danu).

AI usage disclosure

Generative AI tools were used to support the development of the test environment for the `DamageScanner` (in addition to manual tests), the docstrings throughout the package (which were manually checked), and the preliminary versions of Figure 1.

Acknowledgements

This project has received funding from the European Union's Horizon Europe research and innovation programme (Grant Agreement No. 101093854), NWO Dutch Research Council (Veni Grant No. VI.Veni.194.033), the World Bank and the Asian Development Bank.

References

Aznar-Siguan, G., & Bresch, D. N. (2019). CLIMADA v1: A global weather and climate risk assessment platform. *Geoscientific Model Development*, *12*(7), 3085–3097.

Baugh, C., Colonese, J., D'Angelo, C., Dottori, F., Neal, J., Prudhomme, C., & Salamon, P. (2024). *Global river flood hazard maps*. European Commission, Joint Research Centre. https://doi.org/10.2905/JRC.VD32YWG

Bresch, D. N., & Aznar-Siguan, G. (2021). CLIMADA v1. 4.1: Towards a globally consistent adaptation options appraisal tool. *Geoscientific Model Development*, *14*(1), 351–363.

Bruijn, J. A. de, Smilovic, M., Burek, P., Guillaumot, L., Wada, Y., & Aerts, J. C. J. H. (2023). GEB v0.1: A large-scale agent-based socio-hydrological model – simulating 10 million individual farming households in a fully distributed hydrological model. *Geoscientific Model Development*, *16*(9), 2437–2454. https://doi.org/10.5194/gmd-16-2437-2023

De Moel, H., Asselman, N., & Aerts, J. (2012). Uncertainty and sensitivity analysis of coastal flood damage estimates in the west of the netherlands. *Natural Hazards and Earth System Sciences*, *12*(4), 1045–1058.

European Commission. (2023). *Multi-hazard infrastructure risk assessment for climate adaptation (MIRACA)*. https://doi.org/10.3030/101093854

IPCC. (2023). Summary for policymakers. In H. Lee & J. Romero (Eds.), *Climate change 2023: Synthesis report. Contribution of working groups i, II and III to the sixth assessment report of the intergovernmental panel on climate change* (pp. 1–34). IPCC. https://doi.org/10.59327/IPCC/AR6-9789291691647.001

Klijn, F., & Buren, R. van. (2007). *Overstromingsrisico's in nederland in een veranderend klimaat: Verwachtingen, schattingen en berekeningen voor het project nederland later*. WL/Delft Hydraulics Delft.

Koks, E. E., & Haer, T. (2020). A high-resolution wind damage model for europe. *Scientific Reports*, *10*(1), 6866.

Koks, E. E., Rozenberg, J., Zorn, C., Tariverdi, M., Vousdoukas, M., Fraser, S. A., Hall, J. W., & Hallegatte, S. (2019). A global multi-hazard risk analysis of road and railway infrastructure assets. *Nature Communications*, *10*(1), 2677.

Mühlhofer, E., Koks, E. E., Kropf, C. M., Sansavini, G., & Bresch, D. N. (2023). A generalized natural hazard risk modelling framework for infrastructure failure cascades. *Reliability Engineering & System Safety*, *234*, 109194.

Nirandjan, S., Koks, E. E., Ye, M., Pant, R., Van Ginkel, K. C., Aerts, J. C., & Ward, P. J. (2024). Physical vulnerability database for critical infrastructure hazard risk assessments–a

systematic review and data collection. *Natural Hazards and Earth System Sciences*, *24*(12), 4341–4368.

Riedel, L., Kropf, C. M., & Schmid, T. (2024). A module for calibrating impact functions in the climate risk modeling platform CLIMADA. *Journal of Open Source Software*, *9*(99), 6755.

Van Ginkel, K. C. H., Dottori, F., Alfieri, L., Feyen, L., & Koks, E. E. (2021). Flood risk assessment of the european road network. *Natural Hazards and Earth System Sciences*, *21*(3), 1011–1027.

Van Oosterhout, L., Koks, E. E., Van Beukering, P., Schep, S., Tiggeloven, T., Van Manen, S., Knaap, M. van der, Duinmeijer, C., & Buijs, S. (2023). An integrated assessment of climate change impacts and implications on bonaire. *Economics of Disasters and Climate Change*, *7*(2), 147–178.

Ye, M., Ward, P. J., Bloemendaal, N., Nirandjan, S., & Koks, E. E. (2024). Risk of tropical cyclones and floods to power grids in southeast and east asia. *International Journal of Disaster Risk Science*, *15*(4), 494–507.